

Java Inheritance

Foundation Guide #4 of 6 — Java Object-Oriented Programming

A comprehensive reference on inheritance in Java: syntax, design discipline, sealed hierarchies, and FinTech applications. Builds on Guides #1–#3 (Classes & Objects, Encapsulation, Constructors).



DEVELOPED BY TALENCIAGLOBAL



JAVA OOP SERIES



FINTECH FOCUSED

Topic Classification

At a Glance

Topic: Inheritance in Java

Category: Core OOP / Type Hierarchies / Code Reuse

Domain: Java Language → OOP → Subtyping & Specialization

Type: Language feature (extends, super) + design discipline

Difficulty: Beginner (syntax) → Advanced (sealed hierarchies, LSP, avoiding traps)

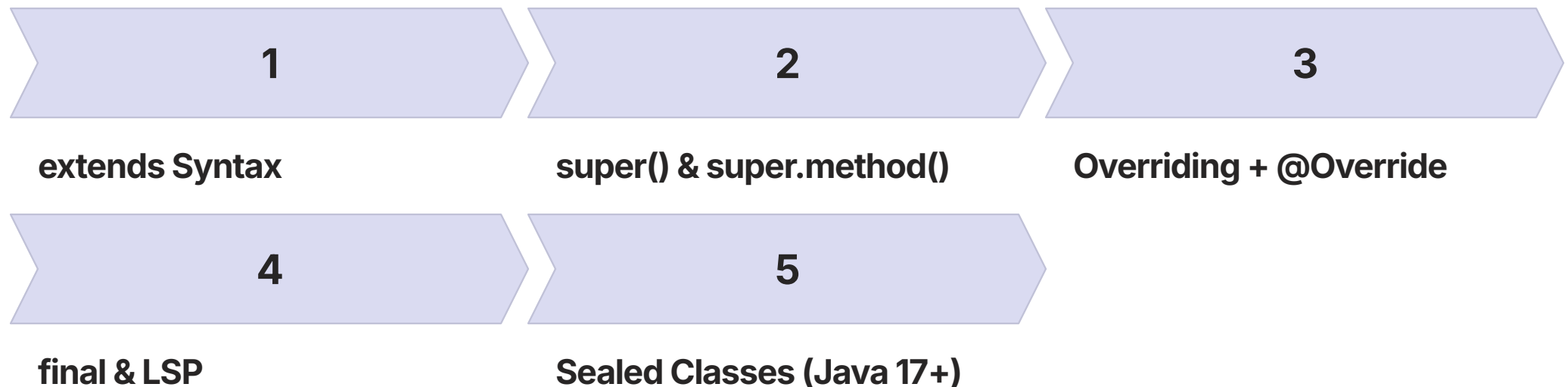
Prerequisites

- Classes & Objects (Guide #1)
- Encapsulation (Guide #2)
- Constructors (Guide #3)
- Method dispatch concepts

Industry Relevance

Mixed — Inheritance is foundational, but **composition is generally preferred** for behavior reuse. Inheritance shines for **closed type hierarchies** (Account → SavingsAccount) when paired with sealed types.

Recommended Learning Path



What Is Inheritance? (Demystified)

Simple intuition: At a bank, all account types — Savings, Current, Loan, NRI — share common behaviors: they have an account number, an owner, a balance; they can be debited and credited. But each *also* has specialized behavior. Savings earns interest, Current has overdraft, Loan accrues monthly EMI.

"Inheritance lets you say: All these account types ARE accounts; here's what they share. Each one ADDS or CHANGES specific things." A SavingsAccount *is-a* Account — wherever you can use an Account, you can use a SavingsAccount.

Model is-a Relationships

SavingsAccount IS an Account — the relationship is genuine and verifiable.

Share Common State

Every account has a number, balance, debit/credit logic — declared once in Account.

Enable Polymorphism

A `List<Account>` can hold any subtype; iteration code stays generic.

Specialize Behavior

`SavingsAccount.applyInterest()` exists only on savings — targeted extension.

Closed Taxonomies

Sealed types model finite business taxonomies that the compiler enforces.

Key Terminology

Mastering inheritance requires fluency with its vocabulary. The following terms appear throughout this guide and in professional Java codebases.

Term	Meaning
Superclass / Parent / Base	The class being extended
Subclass / Child / Derived	The class that extends
<code>extends</code>	Keyword that declares inheritance
<code>super</code>	Reference to the parent (used for constructor or method calls)
Method Overriding	Subclass provides a new implementation of an inherited method
<code>@Override</code>	Annotation that asks the compiler to verify the override
final class	Cannot be extended
final method	Cannot be overridden
<code>Object</code>	Root of every Java class hierarchy
Liskov Substitution Principle (LSP)	Subtypes must be usable wherever supertypes are expected, without breaking behavior
Sealed class/interface (Java 17+)	Limits which classes may extend/implement
Diamond Problem	Multiple inheritance ambiguity — avoided in Java by single class inheritance

Why Does Inheritance Exist?

The Pain It Solves in FinTech

Eliminating Code Duplication

Same `accountNumber`, `balance`, `debit/credit` logic copied across 5 account-type classes → Declare in `Account` once; subclasses inherit automatically.

Generic Processing

Code that processes "any account" must know all types → Polymorphism: `List<Account>` works uniformly across all subtypes.

Differentiated Fee Calculations

Different account types need different fee calculations → Each subclass overrides `calculateFee()` with its own logic.

Extensible Type Dispatch

Adding a new account type requires touching every `if/else` → Add a new subclass; polymorphic dispatch handles routing automatically.



Counterfactual (no inheritance): You'd either copy-paste code across types (duplication = drift = bugs) or stuff everything into one mega-class with `if/else` on a type field — the "God class" anti-pattern. Both are observed in legacy core banking systems and both are widely regretted.

The Dark Side of Inheritance

Inheritance also **causes** significant pain when misused. Understanding these failure modes is as important as understanding the benefits. Industry surveys consistently show that fragile-base-class problems account for a disproportionate share of regression bugs in large Java codebases.

Tight Coupling

Subclass depends on parent internals — the **fragile-base-class problem**. A change to the parent breaks subclasses silently.

Misused for Code Reuse

Inheritance used for code reuse, not is-a relationships, produces confusing hierarchies (e.g., `StackOfPlates extends ArrayList`).

Deep Hierarchies

5+ levels of inheritance are hard to reason about. "Where does this method come from?" becomes a genuine debugging challenge.

Constructor Override Hazard

Overriding methods called from a constructor causes the subclass to observe the parent in mid-initialization — a subtle and dangerous bug.

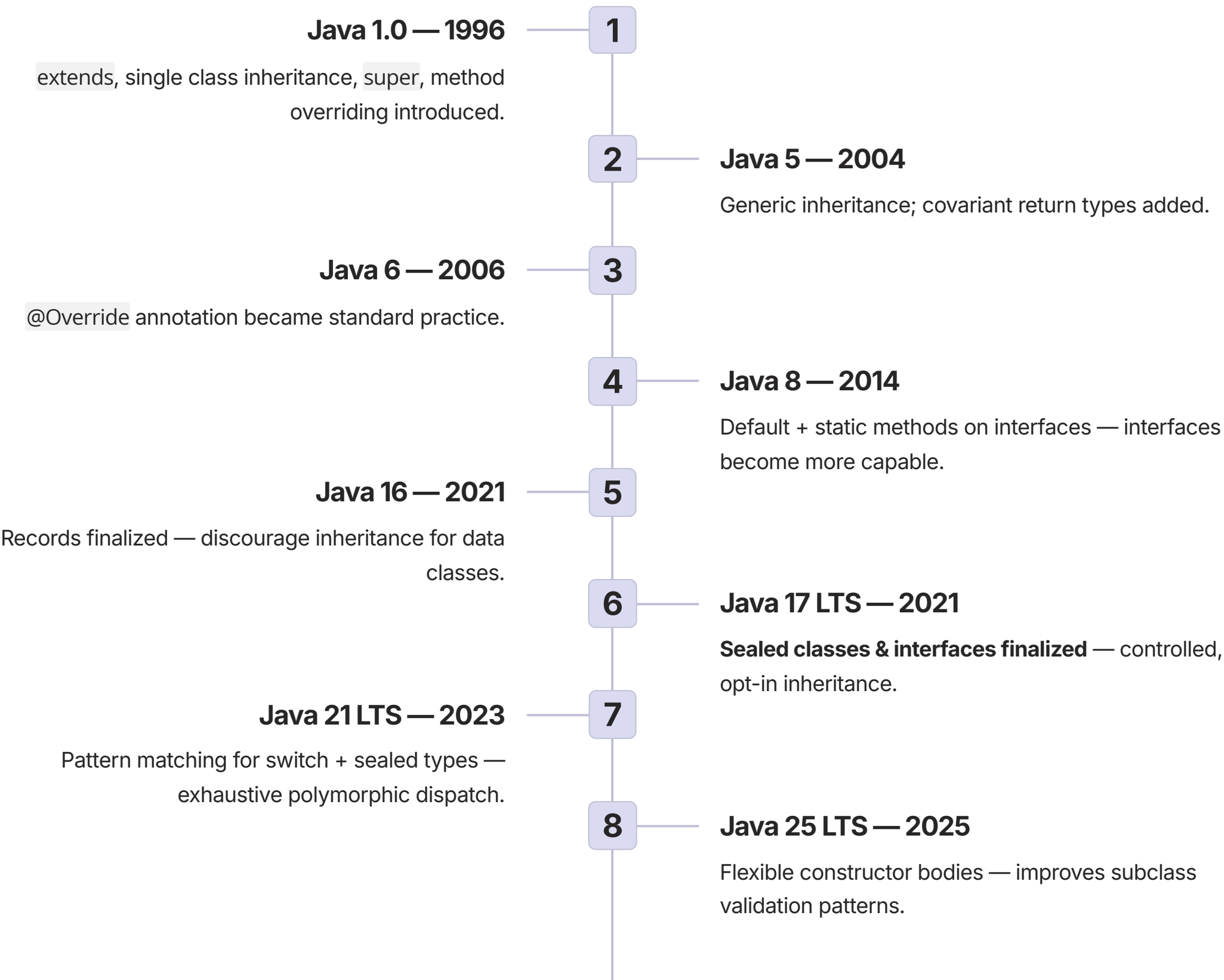
LSP Violations

Mutating parent state without notifying children, or imposing new preconditions, breaks the Liskov Substitution Principle.

This dark side is why modern advice is "favor composition over inheritance" — 1990s OOP overused inheritance → 2000s pushback (Gang of Four, *Effective Java*) → "favor composition" became gospel → sealed classes (Java 17) make inheritance opt-in and constrained.

Evolution & History of Inheritance in Java

Java's approach to inheritance has evolved significantly over nearly three decades, progressively constraining and refining the mechanism to reduce misuse.



Future Trends: Continued de-emphasis of deep inheritance trees in favor of sealed + composition + pattern matching. Value classes (Project Valhalla) will not permit identity-based inheritance.

How Inheritance Works: The Mental Model

Class Hierarchy Structure

A `SavingsAccount` instance contains **all** of `Account`'s fields **plus** its own. Memory layout is a contiguous flat record — there is no separate "parent object" in memory.

Object

▲ (implicit extends)

Account

├── number, owner, balance, status

├── `debit()`, `credit()`, `freeze()`

▲ (extends)

SavingsAccount

├── `interestRate` (subclass-only field)

├── `applyInterest()` (subclass-only)

└── `calculateFee()` `@Override`

Dynamic Dispatch (vtable)

When you call `account.calculateFee()`:

1. The reference type is `Account`, but the actual object may be a `SavingsAccount`.
2. JVM looks up the method via the object's **vtable** (virtual method table).
3. If `SavingsAccount` overrides `calculateFee`, that version runs. Otherwise, the inherited `Account` version runs.

This is **dynamic dispatch** — covered fully in the polymorphism guide (#5).

Initialization Order

01

Memory Allocated

All fields default to zero/null.

03

Subclass Initializers

Instance initializers and field initializers execute.

02

`super(...)` Runs

Parent's constructor executes — recursive up to `Object`.

04

Subclass Constructor Body

The remainder of the subclass constructor executes.

Forms of Inheritance in Java

Java supports several distinct forms of inheritance, each with different rules and use cases. Understanding which form to apply is a core design skill.



Class Inheritance (Single)

`class B extends A`

Example: `SavingsAccount` extends `Account`. Java enforces single class inheritance — no diamond ambiguity.



Interface Implementation (Multiple)

`class A implements I1, I2`

Example: `Card` implements `PaymentMethod`, `Tokenizable`. Multiple capabilities without state coupling.



Interface Inheritance (Multiple)

`interface I3 extends I1, I2`

Example: `ReadWriteRepo` extends `ReadRepo`, `WriteRepo`. Compose interface contracts.



Sealed Inheritance (Constrained)

`sealed class A permits B, C`

Example: `sealed class Account` permits `SavingsAccount`, `CurrentAccount`. Closed, compiler-enforced hierarchy.

Inheritance Modifiers Reference

Modifier	On Class	On Method	Effect
(none)	Extendable	Overridable	Default — usually safe for internal types
<code>final</code>	Not extendable	Not overridable	Locks down the type/method
<code>abstract</code>	Not instantiable	Must be implemented	Forces extension by concrete subclass
<code>sealed</code>	Only permits-listed classes can extend	n/a	Closed hierarchy

Sealed Type Variants (Java 17+)

Sealed classes represent the most significant evolution of Java inheritance since the language's inception. They restore inheritance to its rightful narrow place by making extension **opt-in and constrained**.

sealed

Lists permitted subtypes; the hierarchy is closed. The compiler enforces that all subtypes are listed in the `permits` clause.

non-sealed

A subtype of a sealed class that opens the hierarchy further down. Useful when one branch of a closed taxonomy needs to remain extensible.

final (subtype)

A permitted subtype that cannot be extended further. The most common and recommended choice for leaf classes in a sealed hierarchy.

```
public sealed class Account permits SavingsAccount, CurrentAccount, LoanAccount {}  
public final class SavingsAccount extends Account {} // permitted; locked  
public final class CurrentAccount extends Account {} // permitted; locked  
public non-sealed class LoanAccount extends Account {} // permitted; further extension open
```

- ✅ **Key Benefit:** Sealed hierarchies enable exhaustive pattern matching in switch expressions — the compiler guarantees that all subtypes are handled, eliminating the need for default branches and catching missing cases at compile time.

Real-World FinTech Use Case #1

Account Hierarchy with Sealed Types

A bank's product catalog has Savings, Current, NRI, Loan, and PPI (Prepaid Payment Instrument) accounts. Common attributes: account number, owner, balance, status, debit/credit, audit fields. Specialized: each has different fee structures, interest rules, transaction limits, and regulatory regimes.

The Sealed Abstract Base

```
public sealed abstract class Account
    permits SavingsAccount, CurrentAccount,
        NriAccount, LoanAccount, PpiAccount {

    protected final AccountNumber number;
    protected final Customer owner;
    protected Money balance;
    protected AccountStatus status;

    // Common behavior — final to prevent override
    public final void debit(Money amount) {
        ensureActive();
        if (!canOverdraw() && balance
            .subtract(amount).isNegative())
            throw new InsufficientFundsException(number);
        balance = balance.subtract(amount);
    }

    // Hook methods for subclasses
    protected abstract boolean canOverdraw();
    public abstract Money calculateMonthlyFee();
    public abstract TransactionLimit limit();
}
```

Why This Design Works

- `Account` owns common state + common rules (debit/credit invariants)
- `debit/credit` are `final` — no subclass can break the balance invariant
- Variable behavior is in **template methods** (`calculateMonthlyFee`, `limit`, `canOverdraw`)
- Sealed → exhaustive switch elsewhere knows it covers everything

Outcomes

Adding a new account type forced visible changes in every fee processor and limit checker. Bugs that previously slipped past code review were caught at **compile time**.

- ✓ Sealed + abstract parent + final subclasses + abstract hook methods = the modern Java template-method pattern.

SavingsAccount & CurrentAccount Implementation

SavingsAccount

```
public final class SavingsAccount extends Account {
    private final BigDecimal interestRate;

    public SavingsAccount(AccountNumber n,
        Customer o, Money opening, BigDecimal rate) {
        super(n, o, opening);
        this.interestRate =
            Objects.requireNonNull(rate);
    }

    @Override
    protected boolean canOverdraw() { return false; }

    @Override
    protected void validateOpening(Money opening) {
        if (opening.isNegative())
            throw new IllegalArgumentException(
                "savings opening < 0");
    }

    @Override
    public Money calculateMonthlyFee() {
        return Money.zero(balance.currency());
    }

    public void applyMonthlyInterest() {
        BigDecimal amt = balance.amount()
            .multiply(interestRate)
            .divide(MONTHS, 2, HALF_EVEN);
        credit(new Money(amt, balance.currency()));
    }
}
```

CurrentAccount

```
public final class CurrentAccount extends Account {
    private final Money overdraftLimit;

    public CurrentAccount(AccountNumber n,
        Customer o, Money opening, Money overdraft) {
        super(n, o, opening);
        this.overdraftLimit =
            Objects.requireNonNull(overdraft);
    }

    @Override
    protected boolean canOverdraw() {
        return balance.subtract(overdraftLimit.negate())
            .compareTo(overdraftLimit.negate()) >= 0;
    }

    @Override
    protected void validateOpening(Money opening) {
        /* current allows 0 opening balance */
    }

    @Override
    public Money calculateMonthlyFee() {
        return Money.inr("500");
    }

    @Override
    public TransactionLimit limit() {
        return TransactionLimit.CURRENT_DEFAULT;
    }
}
```

📌 **Design Lesson:** `debit()` and `credit()` are `final` in the parent — the balance invariant is guaranteed regardless of subclass. Variable behavior (fees, limits, overdraft rules) is delegated to abstract hook methods that each subclass must implement.

Real-World FinTech Use Case #2

Inheritance Gone Wrong: A Cautionary Refactor

A payments service had class `CardPayment` extends `Payment`. `Payment` had a `process()` method that did the full flow (auth, capture, persist). `CardPayment` overrode `process()` to add card-specific tokenization. UPI payments later wanted similar customization. Soon, every payment type extended `Payment` and override-tweaked each other's flows. **A change to the parent broke 8 subclasses simultaneously.**

The Problem: Fragile Base Class

- ⊗ Subclasses become coupled to the parent's *implementation*, not just its contract. Refactoring the parent's internal flow breaks subclasses silently.

```
// BEFORE: Fragile inheritance
class CardPayment extends Payment {
  @Override
  public void process() {
    tokenize(); // card-specific
    super.process(); // depends on parent internals
  }
}

// A change to Payment.process() internals
// silently breaks CardPayment, UpiPayment,
// NetBankingPayment, WalletPayment...
```

The Fix: Composition + Sealed + Pattern Matching

```
public sealed interface PaymentMethod
  permits Card, Upi, NetBanking, Wallet { }

public record Card(String token,
  String maskedPan) implements PaymentMethod { }
public record Up (String vpa)
  implements PaymentMethod { }

public final class PaymentProcessor {
  public PaymentResult process(Payment payment) {
    var token = switch (payment.method()) {
      case Card c  -> tokenizer.tokenize(c);
      case Upi u   -> u.vpa();
      case NetBanking nb -> nb.bankCode();
      case Wallet w  -> w.walletId();
    };
    return auth.authorize(payment, token)
      .flatMap(ledger::settle);
  }
}
```

- ✅ **Outcome:** Coupling between payment types eliminated. New payment type = new record + one switch case, not a new subclass override of complex flow logic. **Inheritance for behavior reuse is a trap.**

Real-World FinTech Use Case #3

Exception Hierarchies — One of the Few Genuinely Good Uses

A bank's exception strategy needed: catch-all-banking-errors at the API edge, but also specific catches for retryable network issues, business rule violations, and audit-required compliance failures.

```
// All bank exceptions
public abstract class BankingException
    extends RuntimeException {
    private final String errorCode;
    protected BankingException(String code,
        String msg, Throwable cause) {
        super(msg, cause);
        this.errorCode = code;
    }
}
```

```
// Retryable infrastructure failures
public class TransientBankingException
    extends BankingException { ... }
```

```
// Non-retryable business rule violations
public class BusinessRuleViolation
    extends BankingException { ... }
public class InsufficientFundsException
    extends BusinessRuleViolation { ... }
public class AccountFrozenException
    extends BusinessRuleViolation { ... }
```

```
// Audit-required (regulatory implications)
public class ComplianceFailure
    extends BankingException { ... }
public class SanctionsHitException
    extends ComplianceFailure { ... }
```

Why This Is Good Inheritance

- catch (TransientBankingException) → retry the operation automatically
- catch (BusinessRuleViolation) → return HTTP 422 to the client
- catch (ComplianceFailure) → trigger SAR (Suspicious Activity Report)

Outcomes

Exception handling at the API edge stayed concise. Regulators saw a clear failure taxonomy. Adding a new exception (e.g., `KycExpiredException` extends `BusinessRuleViolation`) automatically inherits handling at every catch site.

- ✓ **Lesson:** Exception hierarchies are one of the few places where deep inheritance reliably pays off. The is-a relationship is genuine and the hierarchy is stable.

Implementation Examples & Best Practices

Example 1 — @Override Discipline

```
// CORRECT — compiler verifies override
public class SavingsAccount extends Account {
    @Override
    public Money calculateMonthlyFee() {
        return Money.zero(balance.currency());
    }
}
```

```
// DANGEROUS — silent failure
public class SavingsAccount extends Account {
    // Typo! Creates NEW method, doesn't override.
    // Parent's calculateMonthlyFee() still wins.
    public Money calculateMonthlyFees() {
        return Money.zero(balance.currency());
    }
}
```

The @Override Rule

Always use

Advantages & Disadvantages

A balanced assessment of inheritance is essential for sound architectural decision-making. The following matrix captures the key trade-offs across six dimensions.

Dimension	Advantages	Disadvantages
Code Reuse	Common code declared once; all subclasses inherit automatically	Coupling: subclass depends on parent internals (fragile base)
Polymorphism	Treat subtypes uniformly via supertype references	Wrong abstractions are hard to undo once established
Modeling	Clear is-a relationships that mirror business domain	Often misused for code reuse where composition is better
Extensibility	Open for new subtypes without modifying existing code	Open by default → undisciplined extension by consumers
Sealed Hierarchies	Closed world; exhaustive dispatch; compiler-enforced	Must update permits clause when adding subtypes
Testing	Mock parent for subclass tests; shared test infrastructure	Deep hierarchies hard to test in isolation

80%

Regression Risk

Of fragile-base-class bugs originate from inheritance misused for code reuse rather than genuine is-a relationships.

5+

Hierarchy Depth Limit

Levels of inheritance beyond which reasoning about method origin becomes impractical in production codebases.

Java 17

Sealed Classes

The LTS release that finalized sealed classes, restoring inheritance to its rightful narrow, controlled place.

Performance Considerations

Virtual Method Dispatch

Method calls on a reference of supertype use vtable lookup. The JIT compiler (C2/Graal) **devirtualizes** monomorphic call sites — when only one actual type is seen at a call site, the JIT converts it to a direct call, often inlined.

Call Site Type	Subtypes Seen	JIT Behavior
Monomorphic	1	Direct call + inline
Bimorphic	2–3	Inline well
Megamorphic	4+	Less optimal path

Sealed types help: Closed world → JIT knows all subtypes → more aggressive devirtualization, especially combined with pattern matching for switch.

Common Performance Pitfalls

Megamorphic Call Sites

Symptom: High latency, poor inlining in hot paths.

Fix: Reduce subtype count; use sealed + pattern matching.

instanceof Chains

Symptom: Hard to optimize; code smell.

Fix: Pattern matching for switch over sealed type.

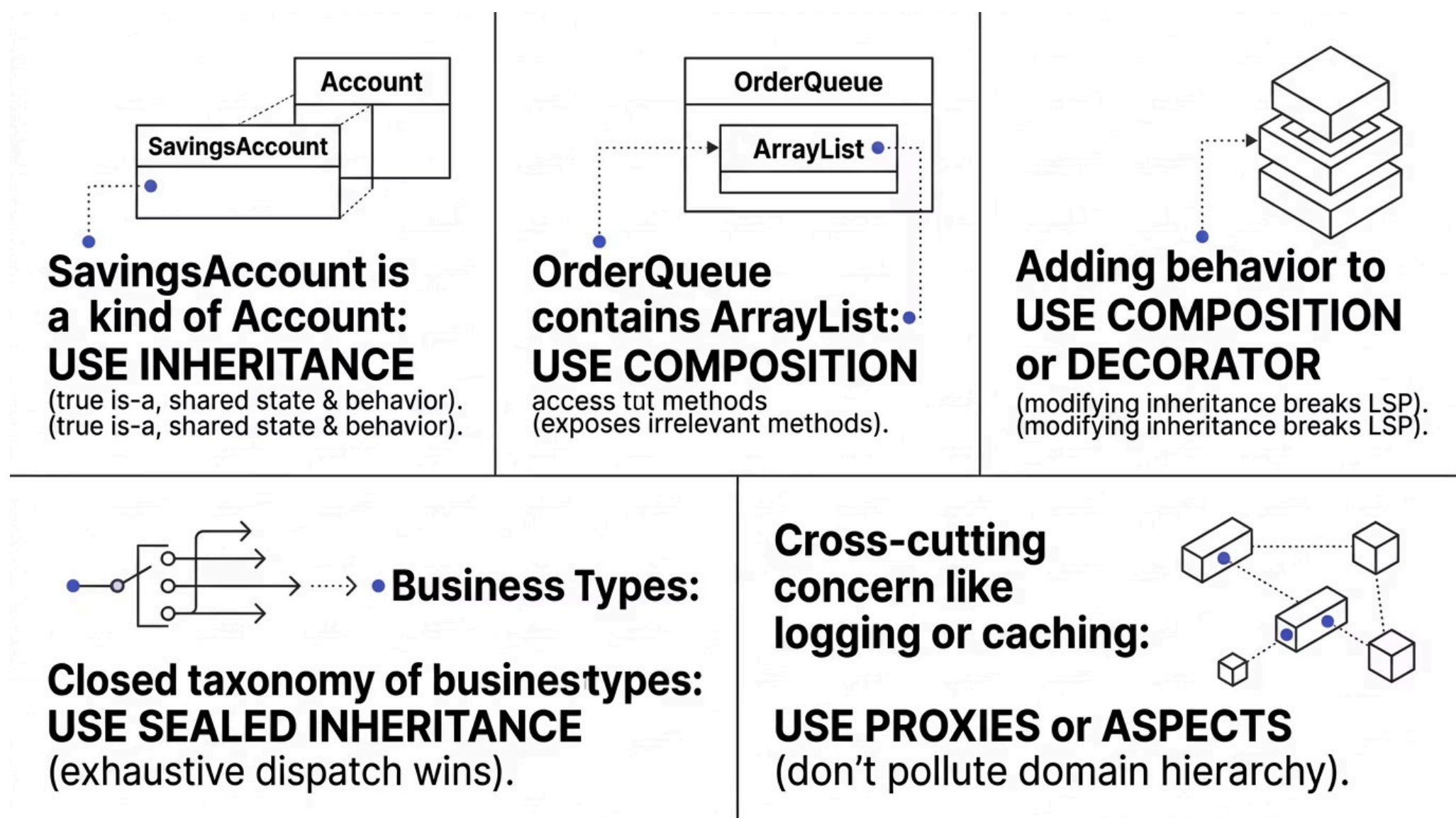
Overridden Methods Preventing Escape Analysis

Symptom: Unexpected allocations.

Fix: Use `final` where possible.

Design & Architectural Considerations

Favor Composition Over Inheritance — The Single Most Important Heuristic



The Composition Pattern — The Alternative

```
// Instead of: class CreditCardAccount extends Account
public final class CreditCardAccount {
    private final Account underlyingAccount; //
    composition
    private final CreditLine creditLine;
    private final CardDetails card;

    public Money availableCredit() {
        return creditLine.limit()
            .subtract(underlyingAccount.balance().negate());
    }

    public void charge(Money amount, String merchant) {
        if (amount.compareTo(availableCredit()) > 0)
            throw new CreditLimitExceededException();
        underlyingAccount.debit(amount);
        card.recordSwipe(merchant);
    }
}
```

Composition Trade-offs

Pros:

- No fragile-base coupling
- Can swap Account implementation
- Can compose multiple capabilities (creditLine + card + rewards)
- Easier to test in isolation

Cons:

- Slightly more boilerplate wiring code
- Doesn't get polymorphic List<Account> substitution for free

The Liskov Substitution Principle

LSP is the cornerstone of correct inheritance design. If a method takes `Account`, you must be able to pass any subclass without surprises. Violating LSP means the inheritance relationship is wrong and must be refactored.

✗ LSP Violation A subclass that throws an exception the parent never threw.	✗ LSP Violation A subclass that imposes new preconditions the parent didn't require.
✗ LSP Violation A subclass that weakens postconditions — does <i>less</i> than the parent guarantees.	✓ LSP Compliant A subclass that adds capabilities without changing parent contracts.

Anti-Patterns to Avoid

Anti-Pattern	Recommended Refactor
Deep hierarchy (5+ levels)	Flatten; use sealed + composition
Extending classes for code reuse only	Composition
Subclass overrides called from parent constructor	Don't override; make method final/private; document if essential
Public concrete base class designed for inheritance ad-hoc	Make abstract or final; or use sealed
Multiple inheritance via clever interface defaults	Use composition; defaults are sugar, not substitute
Inheritance across module boundaries for unrelated types	Almost always wrong; refactor to interface or composition

Security, Compliance & Comparison

Security & Compliance Considerations

Risk	Mitigation
Subclass bypasses parent validation	Mark validation method <code>final</code> or <code>private</code> ; use sealed
Subclass overrides security-sensitive method	Mark <code>final</code> or seal the hierarchy
Deserialization creates subclass bypassing constructor	Use deserialization proxies; avoid Java serialization; records validate via compact constructor
Hidden parent method exposes internals	Audit protected members; prefer <code>private</code> + protected accessors
Untrusted code subclassing exposed types	Make exposed classes <code>final</code> ; use sealed boundaries



Compliance Angle: Sealed hierarchies = enumerable taxonomy — auditors can see the complete set of subtypes. Final security-critical methods guarantee behavior cannot be overridden by malicious or accidental subclasses.

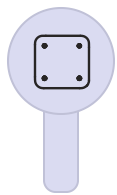
Inheritance vs Alternatives — Quick Reference

Approach	When to Use
Class inheritance	Genuine is-a; shared state + behavior
Sealed inheritance	Closed business taxonomy; exhaustive dispatch
Interface implementation	Capability declaration (Tokenizable, Refundable)
Composition	Behavior reuse without is-a
Delegation	Forward calls to a contained object
Decorator	Add behavior transparently; stackable
Mixins via default methods	Shared default behavior without state

Final Takeaway

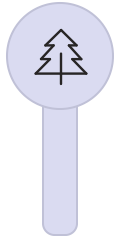
"In FinTech, the wrong inheritance hierarchy is harder to fix than the wrong database schema. Pick is-a, not has-a-uses-a. Seal what's closed. Compose what's flexible. The compiler will be your auditor either way."

The Architect's Discipline in 2026



Default to Composition

For behavior reuse, composition is the default choice. Inheritance is the exception, not the rule.



Use Inheritance for Closed Taxonomies

When modeling closed business taxonomies — Account types, Event types, Exception types — inheritance is the right tool.



Make Hierarchies Sealed

Sealed hierarchies let the compiler enforce closure. Exhaustive switch expressions eliminate entire categories of runtime bugs.



Make Leaf Classes Final

Prevent accidental extension. Design for inheritance or prohibit it — there is no safe middle ground.



Use @Override Always

Type-safe overriding is non-negotiable. Every override must be annotated — no exceptions.



Verify LSP for Every Subclass

Every subclass must be a drop-in replacement for its parent. If it isn't, the inheritance is wrong — refactor immediately.